

SISTEMA DE GESTÃO DE LEADS

1000 Valle Multimarcas

Projeto ABP · MongoDB · Modelagem Não Relacional



- MongoDB

O Desafio

CONTEXTO

1000 Valle Multimarcas
Revendedora multi-unidades.

Leads chegam por:

-  WhatsApp / Instagram
-  Visita presencial
-  Telefone / Formulários

REQUISITOS

- ◆ 6 coleções obrigatórias
- ◆ Embedding + Referencing
- ◆ Inserção mínima de dados
- ◆ Consultas c/ filtro, projeção, sort, paginação
- ◆ Aggregations gerenciais

4 REGRAS DE NEGÓCIO

◆ Lead → Cliente

◆ Lead → Loja + Atendente

◆ 1 negociação ativa / lead

◆ Histórico registrado

As 6 Coleções



clientes

REF

Dados cadastrais do cliente interessado.



leads

REF

Oportunidade comercial gerada a partir do interesse do cliente.



lojas

REF

Unidades físicas da 1000 Valle Multimarcas.



usuarios

REF

Atendentes, gerentes e administradores do sistema.



negociacoes

EMBED

Processo comercial do lead. Histórico embutido (embedding).

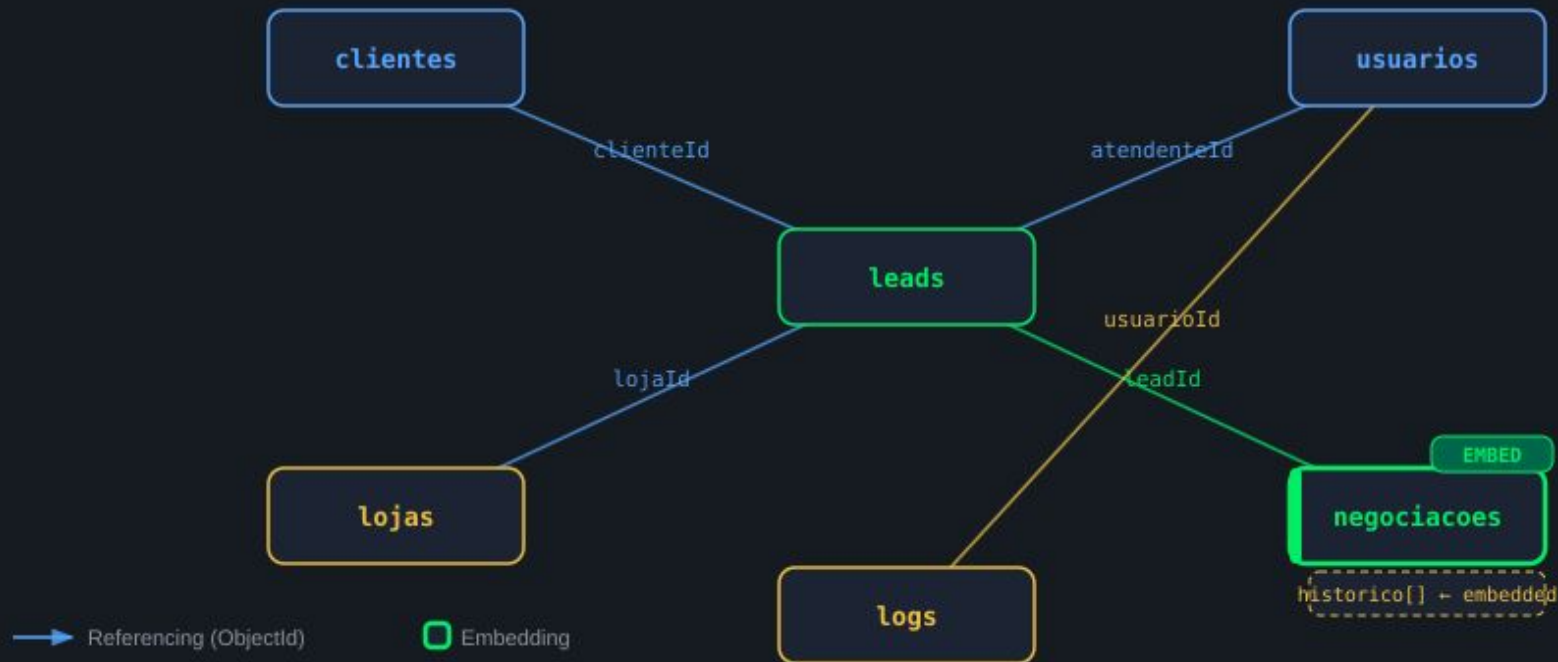


logs

REF

Trilha de auditoria das operações realizadas no sistema.

Mapa de Relacionamentos



Coleção: leads

```
{  
  _id: ObjectId,  
  nomeLead: String,  
  clienteId: ObjectId,    ← REF  
  lojaId: ObjectId,      ← REF  
  atendenteId: ObjectId, ← REF  
  origem: 'whatsapp' | 'visita' ...  
  status: 'novo' | 'em_atendimento' ...  
  importancia: 'alta' | 'media' | 'baixa'  
}
```

clienteId → **clientes**

Dados do cliente atualizados centralmente

lojaId → **lojas**

Permite análise de leads por unidade

atendenteId → **usuarios**

Performance por atendente via aggregation



leads é o hub central: referencia clientes, lojas e atendentes via ObjectId

Coleção: negociacoes

```
{
  _id: ObjectId,
  leadId: ObjectId,    ← REF
  valorProposto: Number,
  status: String,
  ativo: Boolean,
  historico: [         ← EMBEDDED
    {
      data: Date,
      evento: String,
      obs: String
    }
  ]
}
```

EMBEDDING



historico[]

- ◆ Pertence 100% à negociação
- ◆ Lido sempre junto
- ◆ Nunca reutilizado
- ◆ Crescimento previsível

leadId → leads

Rastreia qual negociação pertence a qual lead



Índice parcial único: apenas 1 negociação ativa por lead → `db.negociacoes.createIndex({ leadId:1, ativo:1 }, { unique:true, partialFilterExpression: { ativo:true } })`

Referencing

Documentos ligados por ObjectId

- ◆ Entidades com vida própria
- ◆ Reutilizadas por múltiplos documentos
- ◆ Atualizadas de forma centralizada
- ◆ Evita duplicação de dados cadastrais

 *Vantagem: atualização única reflete em todos os documentos*

```
lead { }
```

```
clienteId: ObjectId → clientes
lojaId: ObjectId → lojas
atendenteId: ObjectId → usuarios
```

```
negociacao { }
```

```
leadId: ObjectId → leads
```

```
log { }
```

```
usuarioId: ObjectId → usuarios
```

Embedding

Documentos dentro de documentos

- ◆ Dados lidos sempre juntos
- ◆ Pertence a um único documento pai
- ◆ Crescimento controlado
- ◆ Sem necessidade de atualização independente

⚡ *Vantagem: uma leitura = dados completos, sem joins*

```
negociacao { }
```

```
leadId: ObjectId  
valorProposto: Number  
status: String
```

```
historico: [  
  { data, evento, obs } ← EMBEDDED
```



negociacoes.historico

Histórico embutido diretamente na negociação

Critério de Decisão

EMBEDDING



`negociacoes.historico`

Por quê?

- Histórico pertence unicamente à negociação
- Lido sempre em conjunto
- Cresce de forma controlada
- Ciclo de vida idêntico ao documento pai

REFERENCING



`leads, clientes, lojas, usuarios, logs`

Por quê?

- Entidades reutilizadas por múltiplos docs
- Dados cadastrais com vida própria
- Atualização centralizada
- Indicadores por loja, atendente, cliente

Dados Inseridos

Coleção	Docs	Destaque	Regra aplicada
 lojas	3	Centro · Norte · Sul	IDs fixos — referenciados por leads e negociações
 usuarios	5	3 atendentes · 1 gerente · 1 admin	Campo cargo define papel no fluxo e na auditoria (logs)
 clientes	5	Dados cadastrais independentes	Entidade independente — referenciada via clienteld
 leads	5	3 origens · 4 status · importância variada	REF para clienteld, lojald e atendenteld — hub central
 negociacoes	2	historico[] ← EMBEDDING	ativo: true garante 1 negociação ativa por lead
 logs	2	criação + atualização de lead	Vincula usuariold + referenciald — rastreabilidade total

Operadores de Consulta

\$and

Lógico

Todas as condições devem ser verdadeiras

```
status novo AND origem whatsapp
```

\$lt

Comparação

Menor que o valor especificado

```
valorProposto < 120000
```

\$or

Lógico

Ao menos uma condição deve ser verdadeira

```
origem whatsapp OR instagram
```

\$exists

Elemento

Verifica se o campo existe no documento

```
email $exists: true
```

\$gt

Comparação

Maior que o valor especificado

```
valorProposto > 30000
```

\$in

Array

Valor está dentro de uma lista definida

```
status $in: ['novo', 'em_atendimento']
```

Consultas na Prática

01 · Leads ativos por canal

```
db.leads.find({  
  
  $and: [  
    { status: {  
      $in: ["novo", "em_atendimento"] } },  
    {  
      $or: [{ origem: "whatsapp" }, { origem: "instagram" }] }  
    ]  
  }  
});
```

\$and

\$or

\$in

- ♦ **\$and** exige ambas as condições
- ♦ **\$or** qualquer das origens
- ♦ **\$in** filtra status de forma compacta

02 · Leads recentes com paginação

```
db.leads.find({}, { nomeLead:1, status:1, origem:1, _id:0 })  
  .sort({ createdAt:-1 })  
  .skip(0).limit(5);
```

projeção

.sort()

.skip/limit

- ♦ **1 = inclui** · **_id:0** = oculta
- ♦ **sort -1** mais recente primeiro
- ♦ **skip(0) limit(5)** página 1

Índices

Índices aceleram consultas frequentes e reduzem o tempo de execução ao evitar varredura completa da coleção.

leads

clienteId: 1

Filtros por cliente

lojaId: 1

Filtros por unidade

atendenteId: 1

Performance por atendente

createdAt: -1

Ordenações por data desc.

negociacoes / logs

negociacoes

leadId: 1

Join negociação → lead

logs

referenciaId: 1

Auditoria por entidade

negociacoes

leadId:1, ativo:1

Índice parcial — 1 ativa/lead



Benefício: leitura direta pelo índice → sem varredura completa → menor tempo de resposta

Pipeline de Aggregation

Aggregations processam documentos em etapas (pipeline) para gerar indicadores, agrupamentos e métricas analíticas.

Leads por Origem

```
$group: { _id: '$origem',  
          total: { $sum: 1 } }  
$sort: { total: -1 }
```

Leads por Status

```
$group: { _id: '$status',  
          total: { $sum: 1 } }  
$sort: { total: -1 }
```

Leads por Atendente

```
$group: { _id: '$atendenteId',  
          totalLeads: { $sum: 1 } }  
$sort: { totalLeads: -1 }
```

Leads por Importância

```
$match: { importancia: { $exists: true } }  
$group: { _id: '$importancia',  
          total: { $sum: 1 } }
```

Atendimento dos Requisitos



Coleções obrigatórias criadas: clientes, leads, usuarios, negociacoes, logs e lojas



Inserção mínima: 5 clientes · 10 leads · 10 negociações · 5 usuários · 10 logs · 3 lojas



Embedding aplicado em negociacoes.historico



Referencing aplicado entre todas as entidades reutilizáveis



Consultas com \$and, \$or, \$gt, \$lt, \$exists, projeção, sort e paginação



Aggregations para indicadores: por origem, status, atendente e importância



Evidências visuais em MongoDB Compass incluídas na documentação

Obrigado!

Sistema de Gestão de Leads · 1000 Valle Multimarcas

Equipe ErrorSquad

Felipe Pacheco · Leonardo Irineu · Carlos Eduardo
João Vitor · Arthur Facchinetti · Caio Araujo



MongoDB · 2026